

CSRF & SSRF Attacks

CSRF: the victim's browser is the weapon

A logged-in user visits an attacker page; the page makes a request to the victim's app *with the victim's cookies*. The app thinks the victim did it.

```
<!-- attacker page -->

<!-- cookie auto-sends; action executes -->
```

Why modern defenses exist

`SameSite=Lax/Strict` cookies (browser won't send on cross-site POST)

`CSRF tokens` (server-issued, per-session, validated)

`Custom headers/origin checks` (double-submit tokens)

SSRF: your server fetches attacker-controlled URLs

An app that exposes a 'fetch this URL' or image/PDF/import feature lets the attacker control WHERE the server connects.

```
# classic cloud meta-data theft (AWS):
GET /import?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/
# internal probing bypassing the firewall:
GET /proxy?url=http://127.0.0.1:8080/admin
GET /proxy?url=dict://user:pass@10.0.0.5:11211/ # gopher/dict protocols
```

Variants

Full SSRF (response returned to attacker) vs blind (only observable via timing/error)

Protocol tricks: http, file://, gopher, redirect-following to bypass URL filters

SSRF defenses

Denylist/allowlist the destinations (allow only needed domains)

Block link-local/loopback/metadata IPs (169.254.169.254, 127.0.0.0/8, 0.0.0.0)

Split network: the fetch service runs egress-restricted, not on trusted DNS

No credentials in the forging layer; redirects disabled unless necessary

Detect: any outbound to metadata addresses = immediate alarm

CSRF is identity-based trust; SSRF is server-side reach. Both break trust boundaries you didn't realize existed.

[← Cross-Site Scripting \(XSS\) D...](#)

[Command Injection, Path Traver...](#)